IMPORTS

In [5]:
```
# Import create_engine (to connect to the database) and text (to write raw SQL queries) from SQLAlchemy.

import pandas as pd
import numpy as np
import os
from sqlalchemy import create_engine, text, event
```

In [6]:
```
pd.set_option('display.max_columns', None)
```

Establishing Connection with SQL Server

In [2]:
```
engine = create_engine(
    r"mssql+pyodbc://DEVANSH\SQLEXPRESS/master?driver=ODBC+Driver+17+for+SQL+Server",
    isolation_level = "AUTOCOMMIT"
)
with engine.connect() as conn:
    conn.execute(text("CREATE DATABASE inventory"))
    print("Database 'inventory' created successfully!")
```

```
Database 'inventory' created successfully!
```

In [5]:
```
engine = create_engine(
    r"mssql+pyodbc://DEVANSH\SQLEXPRESS/inventory?driver=ODBC+Driver+17+for+SQL+Server",
    isolation_level="AUTOCOMMIT"
)

with engine.connect() as conn:
    result = conn.execute(text("SELECT DB_NAME()"))  # Check current DB
    current_db = result.scalar()
    print(f"Connected to database: {current_db}")
```

```
Connected to database: inventory
```

Export CSV to SQL Server

In [ ]:
```
@event.listens_for(engine, "before_cursor_execute")
def enable_fast_insert(conn, cursor, statement, parameters, context, executemany):
    if executemany:
        cursor.fast_executemany = True

# Path to your CSV folder
path = r"D:\Datasets Python\End to End - Vendor Data"

# Ingest CSV file
file = "sales.csv"

full_path = os.path.join(path, file)
table_name = file[:-4]

print(f"\nIngesting {file} into table '{table_name}'...")

first_chunk = True
for chunk in pd.read_csv(full_path, chunksize=2000000):
```

```
    if first_chunk:
        chunk.to_sql(table_name, con=engine, if_exists='replace', index=False)
        first_chunk = False
    else:
        chunk.to_sql(table_name, con=engine, if_exists='append', index=False)

print(f"Done: {table_name}")
```

```
Done: inventory
```

Append Incase previous data already existed

In [ ]:
```
@event.listens_for(engine, "before_cursor_execute")
def enable_fast_insert(conn, cursor, statement, parameters, context, executemany):
    if executemany:
        cursor.fast_executemany = True

# Path to your CSV folder
path = r"D:\Datasets Python\End to End - Vendor Data"
file = "sales.csv"
full_path = os.path.join(path, file)
table_name = file[:-4]

# Skip first 10M rows + 1 for the header (so skip 10,000,001 rows)
skip_rows = 11_585_571

print(f"\nAppending last 2M rows from {file} into table '{table_name}'...")

for chunk in pd.read_csv(full_path, chunksize=500_000, skiprows=range(1, skip_rows), header=0):
    chunk.to_sql(table_name, con=engine, if_exists='append', index=False)

print(f"Appended final rows to '{table_name}' successfully.")
```

```
Appending last 2M rows from sales.csv into table 'sales'...
✅ Appended final rows to 'sales' successfully.
```

Check Tables

In [21]:
```
query = """
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
"""

tables = pd.read_sql(query, engine)
tables
```

Out[21]:

	TABLE_NAME
0	sales
1	begin_inventory
2	end_inventory
3	purchase_prices
4	purchases
5	vendor_invoice

In [57]:
```
for x in tables['TABLE_NAME']:
    query = f"SELECT COUNT(*) FROM [{x}]"
    count = pd.read_sql(query, engine)
    print(f"Count of records in {x}: {count.iloc[0,0]}")
```

```
Count of records in sales: 12825363
Count of records in begin_inventory: 206529
Count of records in end_inventory: 224489
Count of records in purchase_prices: 12261
Count of records in purchases: 2372474
Count of records in vendor_invoice: 5543
```

In [63]:
```
for x in tables['TABLE_NAME']:
    query = f"SELECT TOP 5 * FROM [{x}]"
    top5 = pd.read_sql(query, engine)
    print(f"{x} TOP 5 records: ")
    display(top5)
    print('-'*90)
```

```
sales TOP 5 records:
```

	inventory_id	store_id	brand_id	brand_name	size	quantity	sales_revenue	selling_price	sale_date	volume	classification	excise_tax	vendor_id	vendor_name
0	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL	1	16.49	16.49	2024-01-01	750.0	1	0.79	12546	JIM BEAM BRANDS COMPANY
1	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL	2	32.98	16.49	2024-01-02	750.0	1	1.57	12546	JIM BEAM BRANDS COMPANY
2	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL	1	16.49	16.49	2024-01-03	750.0	1	0.79	12546	JIM BEAM BRANDS COMPANY
3	1_HARDERSFIELD_1004	1	1004	Jim Beam w/2 Rocks Glasses	750mL	1	14.49	14.49	2024-01-08	750.0	1	0.79	12546	JIM BEAM BRANDS COMPANY
4	1_HARDERSFIELD_1005	1	1005	Maker's Mark Combo Pack	375mL 2 Pk	2	69.98	34.99	2024-01-09	375.0	1	0.79	12546	JIM BEAM BRANDS COMPANY

```
------------------------------------------------------------------------------------------
begin_inventory TOP 5 records:
```

	inventory_id	store_id	city	brand_id	brand_name	size	on_hand	price	start_date
0	1_HARDERSFIELD_1000	1	HARDERSFIELD	1000	Goslings Dark'n Stormy VAP	750mL	0	14.990000	2024-01-01
1	1_HARDERSFIELD_1001	1	HARDERSFIELD	1001	Baileys 50mL 4 Pack	50mL 4 Pk	0	5.990000	2024-01-01
2	1_HARDERSFIELD_10021	1	HARDERSFIELD	10021	Clayhouse Syrah Paso Robles	750mL	25	11.990000	2024-01-01
3	1_HARDERSFIELD_1004	1	HARDERSFIELD	1004	Jim Beam w/2 Rocks Glasses	750mL	17	16.490000	2024-01-01
4	1_HARDERSFIELD_1005	1	HARDERSFIELD	1005	Maker's Mark Combo Pack	375mL 2 Pk	7	34.990002	2024-01-01

```
------------------------------------------------------------------------------------------
end_inventory TOP 5 records:
```

	inventory_id	store_id	city	brand_id	brand_name	size	on_hand	price	end_date
0	1_HARDERSFIELD_1001	1	HARDERSFIELD	1001	Baileys 50mL 4 Pack	50mL 4 Pk	0	5.99	2024-12-31
1	1_HARDERSFIELD_10021	1	HARDERSFIELD	10021	Clayhouse Syrah Paso Robles	750mL	23	13.99	2024-12-31
2	1_HARDERSFIELD_1003	1	HARDERSFIELD	1003	Crown Royal VAP Glass+Coastr	750mL	0	22.99	2024-12-31
3	1_HARDERSFIELD_10058	1	HARDERSFIELD	10058	F Coppola Dmd Ivry Cab Svgn	750mL	47	14.99	2024-12-31
4	1_HARDERSFIELD_10062	1	HARDERSFIELD	10062	B de Beauchene CDR Rouge	750mL	29	8.99	2024-12-31

```
------------------------------------------------------------------------------------------
purchase_prices TOP 5 records:
```

	brand_id	brand_name	price	size	volume	classification	purchase_price	vendor_id	vendor_name
0	58	Gekkeikan Black & Gold Sake	12.990000	750mL	750.0	1	9.280000	8320	SHAW ROSS INT L IMP LTD
1	60	Canadian Club 1858 VAP	10.990000	750mL	750.0	1	7.400000	12546	JIM BEAM BRANDS COMPANY
2	61	Margaritaville Silver	13.990000	750mL	750.0	1	10.600000	8004	SAZERAC CO INC
3	62	Herradura Silver Tequila	36.990002	750mL	750.0	1	28.670000	1128	BROWN-FORMAN CORP
4	63	Herradura Reposado Tequila	38.990002	750mL	750.0	1	30.459999	1128	BROWN-FORMAN CORP

---

purchases TOP 5 records:

	inventory_id	store_id	brand_id	brand_name	size	vendor_id	vendor_name	purchase_order_id	purchase_order_date	receiving_date	invoice_date	pay_date	purchase_price	quantity	dollars	cla
0	68_SOLARIS_3840	68	3840	Smirnoff Raspberry Vodka	1.75L	3960	DIAGEO NORTH AMERICA INC	10126	2024-05-09	2024-05-20	2024-05-31	2024-07-04	14.39	5	71.949997	
1	33_HORNSEY_4264	33	4264	Capt Morgan Spiced Rum Trav	750mL	3960	DIAGEO NORTH AMERICA INC	10126	2024-05-09	2024-05-16	2024-05-31	2024-07-04	11.53	11	126.830002	
2	33_HORNSEY_2755	33	2755	Johnnie Walker Red Label	750mL	3960	DIAGEO NORTH AMERICA INC	10126	2024-05-09	2024-05-19	2024-05-31	2024-07-04	19.68	12	236.160004	
3	34_PITMERDEN_3278	34	3278	Gordons London Dry	750mL	3960	DIAGEO NORTH AMERICA INC	10126	2024-05-09	2024-05-20	2024-05-31	2024-07-04	5.79	12	69.480003	
4	33_HORNSEY_3746	33	3746	Gordons Vodka 80 Proof	1.75L	3960	DIAGEO NORTH AMERICA INC	10126	2024-05-09	2024-05-17	2024-05-31	2024-07-04	8.52	4	34.080002	

---

vendor_invoice TOP 5 records:

	vendor_id	vendor_name	invoice_date	purchase_order_id	purchase_order_date	pay_date	quantity	dollars	freight	approval
0	480	BACARDI USA INC	2024-01-12	8106	2023-12-20	2024-02-05	10100	137483.781250	2935.199951	None
1	90046	CANDIA VINEYARDS	2024-01-05	8107	2023-12-20	2024-02-10	24	348.720001	9.080000	None
2	1392	CONSTELLATION BRANDS INC	2024-01-11	8108	2023-12-20	2024-02-10	8466	60281.128906	1549.810059	None
3	1590	DIAGEO CHATEAU ESTATE WINES	2024-01-12	8109	2023-12-20	2024-02-11	2246	14298.089844	408.720001	None
4	3252	E & J GALLO WINERY	2024-01-09	8110	2023-12-20	2024-02-19	8086	56493.230469	1300.920044	None

---

Key Observations From above Tables

- The **Purchase_prices table** shows the purchase price and actual price of the brand
- The **Purchases table** contains actual purchase data, including the date of purchase, products (brands) purchased by vendors, the amount paid (in dollars), and the quantity purchased.

- The purchase price column in **Purchases table** is derived from the **purchase_prices table**, which provides product-wise actual and purchase prices. The combination of vendor and brand is unique in this table.
- The **vendor_invoice table** aggregates data from the **purchases table**, summarizing quantity and dollar amounts, along with an additional column for freight. Each vendor has a unique vendor_id and purchase_order_id.
- The **sales table** captures actual sales transactions, detailing the brands purchased by vendors, the quantity sold, the selling price, and the revenue earned.
- As observed in Database, we wont be using **begin and end inventory tables** due to insufficient data.

Created an Aggregated Table in SQL Server as supplier_performance

- This aggregated table consolidates **vendor-brand level** metrics from purchases, sales, and invoice data.
- It captures key KPIs such as **purchase quantity, unit cost, sales revenue, selling price, excise tax, and freight costs**.

In [100…
```
tables
```

Out[100…

	TABLE_NAME
0	sales
1	supplier_performance_evaluation
2	begin_inventory
3	end_inventory
4	purchase_prices
5	purchases
6	vendor_invoice

Analyzing Aggregated Table

In [6]:
```
query = """
SELECT * FROM supplier_performance_evaluation
ORDER BY total_purchases_amount DESC
"""
supplier_performance = pd.read_sql(query, engine)
```

In [7]:
```
supplier_performance.head(3)
```

Out[7]:

volume	actual_price_per_unit	purchased_price_per_unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs
1750.0	36.99	26.27	145080	3811251.60	34.99	142049.0	5101919.51	672819.31	260999.20	68601.68
1750.0	28.99	23.19	164038	3804041.22	27.95	160247.0	4819073.49	561512.37	294438.66	144929.24
1750.0	24.99	18.24	187407	3418303.68	23.49	187140.0	4538120.60	461140.15	343854.07	123780.22

DATA CLEANING

In [8]:
```
supplier_performance.shape
```

Out[8]:
```
(11101, 15)
```

In [9]:
```
supplier_performance.dtypes
```

Out[9]:
```
vendor_id                   int64
vendor_name                object
brand_id                    int64
brand_name                 object
volume                    float64
actual_price_per_unit     float64
purchased_price_per_unit  float64
total_qty_purchased         int64
total_purchases_amount    float64
selling_price_per_unit    float64
total_qty_sold            float64
total_sales_amount        float64
total_sales_price         float64
total_excise_tax          float64
total_freight_costs       float64
dtype: object
```

The data types seems to be fine.

In [10]:
```
supplier_performance.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11101 entries, 0 to 11100
Data columns (total 15 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   vendor_id                 11101 non-null  int64  
 1   vendor_name               11101 non-null  object 
 2   brand_id                  11101 non-null  int64  
 3   brand_name                11101 non-null  object 
 4   volume                    11101 non-null  float64
 5   actual_price_per_unit     11101 non-null  float64
 6   purchased_price_per_unit  11101 non-null  float64
 7   total_qty_purchased       11101 non-null  int64  
 8   total_purchases_amount    11101 non-null  float64
 9   selling_price_per_unit    10917 non-null  float64
 10  total_qty_sold            10917 non-null  float64
 11  total_sales_amount        10917 non-null  float64
 12  total_sales_price         10917 non-null  float64
 13  total_excise_tax          10917 non-null  float64
 14  total_freight_costs       11101 non-null  float64
dtypes: float64(10), int64(3), object(2)
memory usage: 1.3+ MB
```

In [11]:
```
supplier_performance.isna().sum()
```

Out[11]:
```
vendor_id                    0
vendor_name                  0
brand_id                     0
brand_name                   0
volume                       0
actual_price_per_unit        0
purchased_price_per_unit     0
total_qty_purchased          0
total_purchases_amount       0
selling_price_per_unit     184
total_qty_sold             184
total_sales_amount         184
total_sales_price          184
total_excise_tax           184
total_freight_costs          0
dtype: int64
```

Missing values observed. Here, The total_qty_sold have null values because some products might not have been sold after they were purchased. Hence, we fill all missing values wih 0.

In [12]:
```
supplier_performance.fillna(0, inplace=True)
```

In [13]:
```
supplier_performance.isna().sum()
```

Out[13]:
```
vendor_id                   0
vendor_name                 0
brand_id                    0
brand_name                  0
volume                      0
actual_price_per_unit       0
purchased_price_per_unit    0
total_qty_purchased         0
total_purchases_amount      0
selling_price_per_unit      0
total_qty_sold              0
total_sales_amount          0
total_sales_price           0
total_excise_tax            0
total_freight_costs         0
dtype: int64
```

In [14]:
```
supplier_performance.duplicated().sum()
```

Out[14]:
```
np.int64(0)
```

In [15]:
```
supplier_performance.columns
```

Out[15]:
```
Index(['vendor_id', 'vendor_name', 'brand_id', 'brand_name', 'volume',
       'actual_price_per_unit', 'purchased_price_per_unit',
       'total_qty_purchased', 'total_purchases_amount',
       'selling_price_per_unit', 'total_qty_sold', 'total_sales_amount',
       'total_sales_price', 'total_excise_tax', 'total_freight_costs'],
      dtype='object')
```

In [16]:
```
supplier_performance['vendor_name'].unique()[:20]
```

Out[16]:
```
array(['BROWN-FORMAN CORP          ', 'MARTIGNETTI COMPANIES',
       'PERNOD RICARD USA          ', 'DIAGEO NORTH AMERICA INC   ',
       'BACARDI USA INC            ', 'JIM BEAM BRANDS COMPANY    ',
       'MAJESTIC FINE WINES        ', 'ULTRA BEVERAGE COMPANY LLP ',
       'STOLI GROUP,(USA) LLC      ', 'PROXIMO SPIRITS INC.       ',
       'MOET HENNESSY USA INC      ', 'CAMPARI AMERICA            ',
       'SAZERAC CO INC             ', 'CONSTELLATION BRANDS INC   ',
       'M S WALKER INC             ', 'SAZERAC NORTH AMERICA INC. ',
       'PALM BAY INTERNATIONAL INC ', 'REMY COINTREAU USA INC     ',
       'SIDNEY FRANK IMPORTING CO  ', 'E & J GALLO WINERY         '],
      dtype=object)
```

Remove all the extra spaces in the string

In [17]:
```
supplier_performance['vendor_name'] = supplier_performance['vendor_name'].str.split().str.join(' ')
```

In [18]:
```
supplier_performance['vendor_name'].unique()[:20]
```

Out[18]:
```
array(['BROWN-FORMAN CORP', 'MARTIGNETTI COMPANIES', 'PERNOD RICARD USA',
       'DIAGEO NORTH AMERICA INC', 'BACARDI USA INC',
       'JIM BEAM BRANDS COMPANY', 'MAJESTIC FINE WINES',
       'ULTRA BEVERAGE COMPANY LLP', 'STOLI GROUP,(USA) LLC',
       'PROXIMO SPIRITS INC.', 'MOET HENNESSY USA INC', 'CAMPARI AMERICA',
       'SAZERAC CO INC', 'CONSTELLATION BRANDS INC', 'M S WALKER INC',
       'SAZERAC NORTH AMERICA INC.', 'PALM BAY INTERNATIONAL INC',
       'REMY COINTREAU USA INC', 'SIDNEY FRANK IMPORTING CO',
       'E & J GALLO WINERY'], dtype=object)
```

In [19]:
```
supplier_performance['brand_name'].unique()[:20]
```

Out[19]:
```
array(['Jack Daniels No 7 Black', "Tito's Handmade Vodka",
       'Absolut 80 Proof', 'Capt Morgan Spiced Rum', 'Ketel One Vodka',
       'Grey Goose Vodka', 'Jameson Irish Whiskey', 'Smirnoff Traveler',
       'Tanqueray', 'Jim Beam', 'Johnnie Walker Red Label',
       'Dewars White Label', 'Kendall Jackson Chard Vt RSV',
       'Patron Silver Tequila', 'Johnnie Walker Black Label', 'Kahlua',
       'Baileys Irish Cream', 'Crown Royal',
       'Capt Morgan Original Barrel', 'Bombay Sapphire Gin'], dtype=object)
```

No unwanted spaces were found, but we clean the data as a precaution.

In [20]:
```
supplier_performance['brand_name'] = supplier_performance['brand_name'].str.split().str.join(' ')
```

In [21]:
```
supplier_performance['brand_name'].unique()[:20]
```

Out[21]:
```
array(['Jack Daniels No 7 Black', "Tito's Handmade Vodka",
       'Absolut 80 Proof', 'Capt Morgan Spiced Rum', 'Ketel One Vodka',
       'Grey Goose Vodka', 'Jameson Irish Whiskey', 'Smirnoff Traveler',
       'Tanqueray', 'Jim Beam', 'Johnnie Walker Red Label',
       'Dewars White Label', 'Kendall Jackson Chard Vt RSV',
       'Patron Silver Tequila', 'Johnnie Walker Black Label', 'Kahlua',
       'Baileys Irish Cream', 'Crown Royal',
       'Capt Morgan Original Barrel', 'Bombay Sapphire Gin'], dtype=object)
```

DATA CLEANING SUMMARY

- Checked the dataset shape
- Reviewed data types and found them appropriate
- Identified and filled missing values
- Verified no duplicate records were present
- Removed unwanted spaces from text columns

FEATURE ENGINEERING

In [22]:
```
supplier_performance.head(2)
```

Out[22]:

volume	actual_price_per_unit	purchased_price_per_unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs
1750.0	36.99	26.27	145080	3811251.60	34.99	142049.0	5101919.51	672819.31	260999.20	68601.68
1750.0	28.99	23.19	164038	3804041.22	27.95	160247.0	4819073.49	561512.37	294438.66	144929.24

1) GROSS AMOUNT

a) Gross Profit = Total Sales - COGS (Total Purchases)

In [23]:
```
supplier_performance['gross_profit/loss_amount'] = supplier_performance['total_sales_amount'] - supplier_performance['total_purchases_amount']
```

In [25]:
```
supplier_performance['gross_profit/loss_status'] = supplier_performance['gross_profit/loss_amount'].apply(
    lambda x: 'Profit' if x > 0 else ('Break-even' if x == 0 else 'Loss')
)
```

In [57]:
```
supplier_performance.head(2)
```

Out[57]:

unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status
26.27	145080	3811251.60	34.99	142049.0	5101919.51	672819.31	260999.20	68601.68	1290667.91	Profit
23.19	164038	3804041.22	27.95	160247.0	4819073.49	561512.37	294438.66	144929.24	1015032.27	Profit

b) Gross Profit Margin = (Gross Profit / Total Sales) * 100

In [ ]:
```
supplier_performance['gross_profit_margin'] = np.where(supplier_performance['gross_profit/loss_amount'] > 0,
(supplier_performance['gross_profit/loss_amount'] / supplier_performance['total_sales_amount']) * 100, 0)
```

In [59]:
```
supplier_performance.head(2)
```

Out[59]:

ased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status	gross_profit_margin
5080	3811251.60	34.99	142049.0	5101919.51	672819.31	260999.20	68601.68	1290667.91	Profit	25.297693
4038	3804041.22	27.95	160247.0	4819073.49	561512.37	294438.66	144929.24	1015032.27	Profit	21.062810

2) NET AMOUNT

a) Net Profit = Gross Profit - All Expenses

In [60]:
```
supplier_performance['net_profit/loss_amount'] = (supplier_performance['gross_profit/loss_amount'] - supplier_performance['total_excise_tax'] - supplier_performance['total_freight_costs'])
```

In [62]:
```
supplier_performance['net_profit/loss_status'] = supplier_performance['net_profit/loss_amount'].apply(
    lambda x: 'Profit' if x > 0 else ('Break-even' if x == 0 else 'Loss')
)
```

In [66]:
```
supplier_performance.head(2)
```

Out[66]:

r_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status	gross_profit_margin	net_profit/loss_amount	net_profit/loss_status
34.99	142049.0	5101919.51	672819.31	260999.20	68601.68	1290667.91	Profit	25.297693	961067.03	Profit
27.95	160247.0	4819073.49	561512.37	294438.66	144929.24	1015032.27	Profit	21.062810	575664.37	Profit

b) Net Profit Margin = (Net Profit / Total Sales) * 100

In [ ]:
```
supplier_performance['net_profit_margin'] = np.where(supplier_performance['net_profit/loss_amount'] > 0,
(supplier_performance['net_profit/loss_amount'] / supplier_performance['total_sales_amount']) * 100, 0)
```

In [98]:
```
supplier_performance.head(2)
```

Out[98]:

d	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status	gross_profit_margin	net_profit/loss_amount	net_profit/loss_status	net_profit_margin
.0	5101919.51	672819.31	260999.20	68601.68	1290667.91	Profit	25.297693	961067.03	Profit	18.837362
.0	4819073.49	561512.37	294438.66	144929.24	1015032.27	Profit	21.062810	575664.37	Profit	11.945540

3) **SALES-to-PURCHASE QUANTITY RATIO** = Total Sales Qty / Total Purchase Qty

In [100…
```
supplier_performance['stock_turnover'] = supplier_performance['total_qty_sold'] / supplier_performance['total_qty_purchased']
```

In [101…
```
supplier_performance.head(2)
```

Out[101...

nount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status	gross_profit_margin	net_profit/loss_amount	net_profit/loss_status	net_profit_margin	stock_turnover
919.51	672819.31	260999.20	68601.68	1290667.91	Profit	25.297693	961067.03	Profit	18.837362	0.979108
073.49	561512.37	294438.66	144929.24	1015032.27	Profit	21.062810	575664.37	Profit	11.945540	0.976890

4) SALES-to-PURCHASE PRICE RATIO = Total Sales Price / Total Purchase Price

In [ ]:
```
supplier_performance['stock_amount_turnover'] = supplier_performance['total_sales_amount'] / supplier_performance['total_purchases_amount']
```

In [115...
```
supplier_performance.head(2)
```

Out[115...

_excise_tax	total_freight_costs	gross_profit/loss_amount	gross_profit/loss_status	gross_profit_margin	net_profit/loss_amount	net_profit/loss_status	net_profit_margin	stock_turnover	sales_to_purchase_price_ratio
260999.20	68601.68	1290667.91	Profit	25.297693	961067.03	Profit	18.837362	0.979108	1.338647
294438.66	144929.24	1015032.27	Profit	21.062810	575664.37	Profit	11.945540	0.976890	1.266830

In [125...
```
supplier_performance.rename(columns={'gross_profit/loss_status': 'gross_profit_loss_status', 'net_profit/loss_status': 'net_profit_loss_status', 'gross_profit/loss_amount': 'gross_profit_loss
```

In [112...
```
supplier_performance.shape
```

Out[112...
```
(11101, 23)
```

FEATURE ENGINEERING SUMMARY

- **Gross Profit/Loss Amount:** Calculated as the difference between total sales and total purchases.
- **Gross Profit/Loss Status:** Categorized as Profit (if gross profit > 0), Break-even (if = 0), or Loss (if < 0).
- **Gross Profit Margin:** Computed as gross profit divided by total sales.
- **Net Profit/Loss Amount:** Derived by subtracting all expenses (e.g., excise tax, freight) from gross profit.
- **Net Profit/Loss Status:** Labeled as Profit (if net profit > 0) or Loss (if < 0).
- **Net Profit Margin:** Calculated as net profit divided by total sales.
- **Sales-to-Purchase Quantity Ratio:** Indicates stock movement efficiency; approximates stock turnover.
- **Sales-to-Purchase Price Ratio:** Shows pricing efficiency; a value > 1 indicates profitability.

EXPORT FINAL DATAFRAME TO SQL SERVER

CREATING A TABLE FOR SQL SERVER

In [7]:
```
engine = create_engine(
    r"mssql+pyodbc://DEVANSH\SQLEXPRESS/inventory?driver=ODBC+Driver+17+for+SQL+Server",
    isolation_level="AUTOCOMMIT"
)
```

In [126…
```
supplier_performance.columns
```

Out[126…
```
Index(['vendor_id', 'vendor_name', 'brand_id', 'brand_name', 'volume',
       'actual_price_per_unit', 'purchased_price_per_unit',
       'total_qty_purchased', 'total_purchases_amount',
       'selling_price_per_unit', 'total_qty_sold', 'total_sales_amount',
       'total_sales_price', 'total_excise_tax', 'total_freight_costs',
       'gross_profit_loss_amount', 'gross_profit_loss_status',
       'gross_profit_margin', 'net_profit_loss_amount',
       'net_profit_loss_status', 'net_profit_margin', 'stock_turnover',
       'sales_to_purchase_price_ratio'],
      dtype='object')
```

In [128…
```
final_table_query = """
CREATE TABLE supplier_performance_summary (
    vendor_id INT,
    vendor_name VARCHAR(100),
    brand_id INT,
    brand_name VARCHAR(100),
    volume INT,
    actual_price_per_unit DECIMAL (10,2),
    purchased_price_per_unit DECIMAL (10,2),
    total_qty_purchased INT,
    total_purchases_amount DECIMAL (15,2),
    selling_price_per_unit DECIMAL (10,2),
    total_qty_sold INT,
    total_sales_amount DECIMAL (15,2),
    total_sales_price DECIMAL (15,2),
    total_excise_tax DECIMAL (15,2),
    total_freight_costs DECIMAL (15,2),
    gross_profit_loss_amount DECIMAL (15,2),
    gross_profit_loss_status VARCHAR(20),
    gross_profit_margin DECIMAL (15,2),
    net_profit_loss_amount DECIMAL (15,2),
    net_profit_loss_status VARCHAR(20),
    net_profit_margin DECIMAL (15,2),
    stock_turnover DECIMAL (10,2),
    sales_to_purchase_price_ratio DECIMAL (10,2),
    PRIMARY KEY (vendor_id, brand_id)
);
"""

with engine.connect() as conn:
    conn.execute(text(final_table_query))
```

In [129…
```
query = "SELECT * FROM supplier_performance_summary"
pd.read_sql(query, engine)
```

Out[129…

	vendor_id	vendor_name	brand_id	brand_name	volume	actual_price_per_unit	purchased_price_per_unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount

INGESTING DATA TO TABLE

In [130…
```
supplier_performance.to_sql("supplier_performance_summary", con=engine, index=False, if_exists="replace")
```

Out[130… 90

In [8]:
```
query = "SELECT * FROM supplier_performance_summary"
supplier_performance = pd.read_sql(query, engine)
```

In [101…
```
import matplotlib.pyplot as plt
import seaborn as sns
import math
import warnings
warnings.filterwarnings('ignore')
```

EDA ON FINAL TABLE

In [10]:
```
supplier_performance.head(1)
```

Out[10]:

	vendor_id	vendor_name	brand_id	brand_name	volume	actual_price_per_unit	purchased_price_per_unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount
0	1128	BROWN-FORMAN CORP	1233	Jack Daniels No 7 Black	1750.0	36.99	26.27	145080	3811251.6	34.99	142049.0	5101919.51

In [18]:
```
supplier_performance.columns
```

Out[18]:
```
Index(['vendor_id', 'vendor_name', 'brand_id', 'brand_name', 'volume',
       'actual_price_per_unit', 'purchased_price_per_unit',
       'total_qty_purchased', 'total_purchases_amount',
       'selling_price_per_unit', 'total_qty_sold', 'total_sales_amount',
       'total_sales_price', 'total_excise_tax', 'total_freight_costs',
       'gross_profit_loss_amount', 'gross_profit_loss_status',
       'gross_profit_margin', 'net_profit_loss_amount',
       'net_profit_loss_status', 'net_profit_margin', 'stock_turnover',
       'sales_to_purchase_price_ratio'],
      dtype='object')
```

In [22]:
```
supplier_performance.describe().T
```

Out[22]:

	count	mean	std	min	25%	50%	75%	max
vendor_id	11101.0	10329.175930	18478.053301	2.00	3252.00	7153.00	9552.00	201359.00
brand_id	11101.0	18254.831006	12600.526806	58.00	6206.00	19070.00	25570.00	90631.00
volume	11101.0	844.887848	654.811062	50.00	750.00	750.00	750.00	20000.00
actual_price_per_unit	11101.0	36.629352	148.534699	0.49	10.99	15.99	28.99	7499.99
purchased_price_per_unit	11101.0	25.008177	109.078281	0.36	6.88	10.52	19.72	5681.81
total_qty_purchased	11101.0	3077.337807	10916.912273	1.00	34.00	250.00	1941.00	337660.00
total_purchases_amount	11101.0	29443.158015	120964.323734	0.71	446.16	3556.44	20025.94	3811251.60
selling_price_per_unit	11101.0	33.587459	126.334146	0.00	9.99	14.99	26.99	5799.99
total_qty_sold	11101.0	3015.154761	10777.015741	0.00	31.00	248.00	1879.00	334939.00
total_sales_amount	11101.0	41366.692946	164832.865883	0.00	725.67	5124.79	27534.38	5101919.51
total_sales_price	11101.0	18412.910834	44281.097727	0.00	289.72	2777.91	15665.36	672819.31
total_excise_tax	11101.0	1715.121743	10776.046210	0.00	4.59	44.10	396.60	368242.80
total_freight_costs	11101.0	59170.326662	60912.767016	0.09	11595.97	48347.26	79528.99	257032.07
gross_profit_loss_amount	11101.0	11923.534930	45490.515187	-52002.78	53.16	1389.35	8515.59	1290667.91
gross_profit_margin	11101.0	31.237934	24.744152	0.00	13.42	30.65	40.20	99.72
net_profit_loss_amount	11101.0	-48961.913475	66981.237999	-272403.27	-72377.34	-28812.17	-6089.99	961067.03
net_profit_margin	11101.0	3.358327	10.979329	0.00	0.00	0.00	0.00	98.76
stock_turnover	11101.0	1.722364	5.985775	0.00	0.81	0.98	1.04	274.50
sales_to_purchase_price_ratio	11101.0	2.530739	8.428293	0.00	1.16	1.44	1.67	352.93

DISTRIBUTION PLOTS - To check the range of values for each numerical column and also to get an idea of outliers.

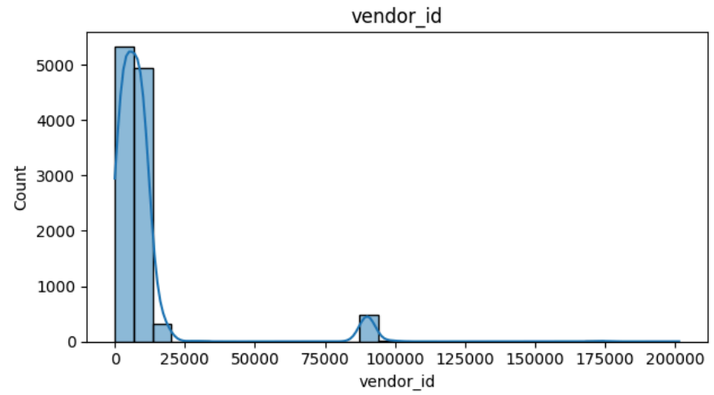
In [28]:
```
# To get only numerical columns
numerical_cols = supplier_performance.select_dtypes(include=np.number).columns
numerical_cols
```

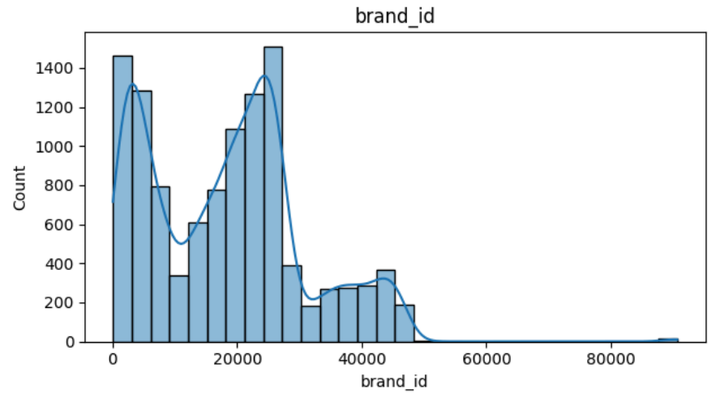
Out[28]:
```
Index(['vendor_id', 'brand_id', 'volume', 'actual_price_per_unit',
       'purchased_price_per_unit', 'total_qty_purchased',
       'total_purchases_amount', 'selling_price_per_unit', 'total_qty_sold',
       'total_sales_amount', 'total_sales_price', 'total_excise_tax',
       'total_freight_costs', 'gross_profit_loss_amount',
       'gross_profit_margin', 'net_profit_loss_amount', 'net_profit_margin',
       'stock_turnover', 'sales_to_purchase_price_ratio'],
      dtype='object')
```

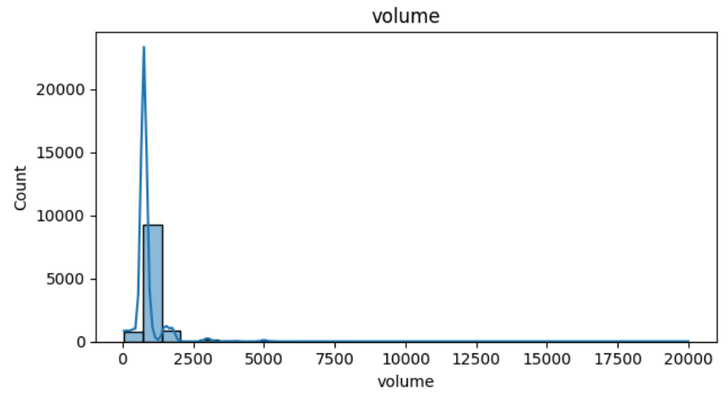
In [52]:
```
for index, column in enumerate(numerical_cols):
    print(index, column)
```

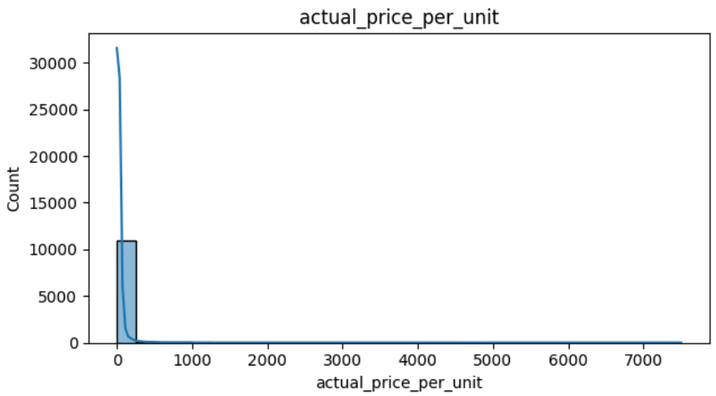
```
0 vendor_id
1 brand_id
2 volume
3 actual_price_per_unit
4 purchased_price_per_unit
5 total_qty_purchased
6 total_purchases_amount
7 selling_price_per_unit
8 total_qty_sold
9 total_sales_amount
10 total_sales_price
11 total_excise_tax
12 total_freight_costs
13 gross_profit_loss_amount
14 gross_profit_margin
15 net_profit_loss_amount
16 net_profit_margin
17 stock_turnover
18 sales_to_purchase_price_ratio
```

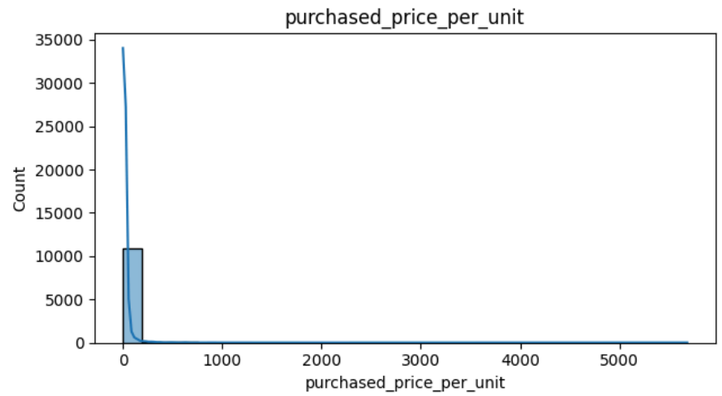
In [57]:
```
plt.figure(figsize=(20,25))

for i, col in enumerate(numerical_cols):
    plt.subplot(7, 3, i + 1)
    sns.histplot(supplier_performance[col], kde=True, bins=30)
    plt.title(col)

plt.tight_layout()
plt.show()
```

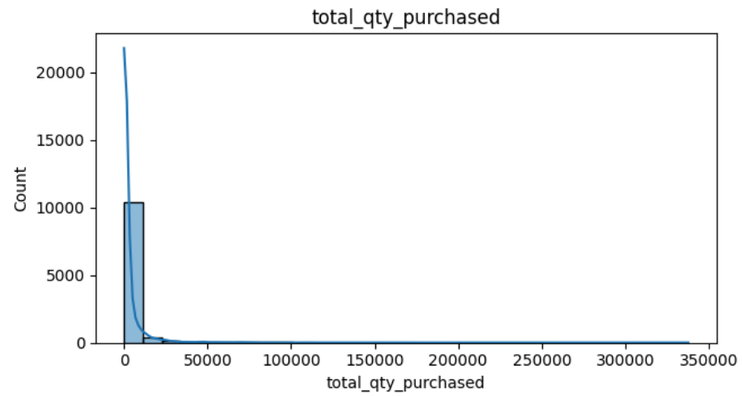

vendor_id
Count
vendor_id

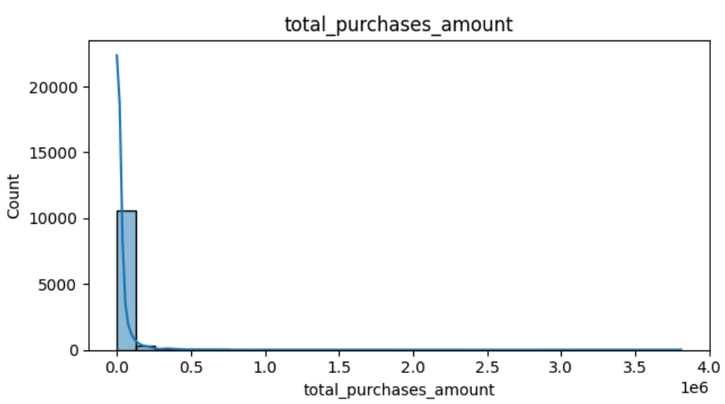

brand_id
Count
brand_id

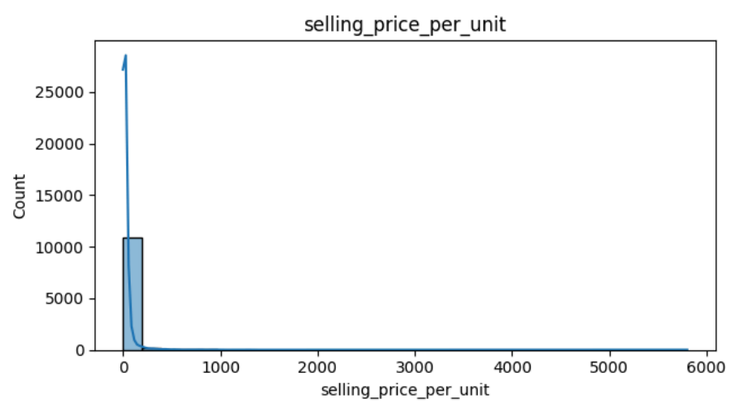

volume
Count
volume

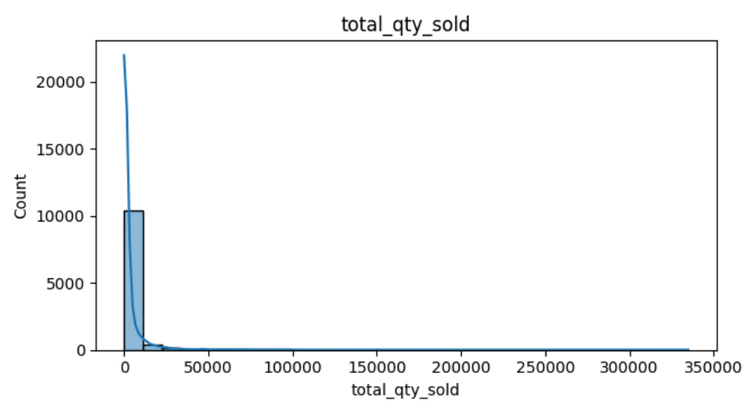

actual_price_per_unit
Count
actual_price_per_unit

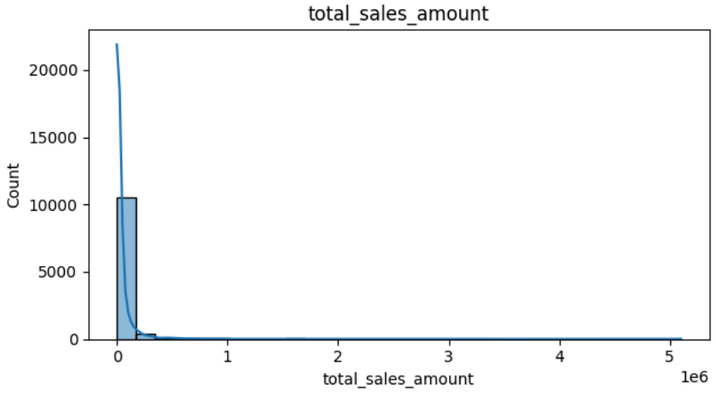

purchased_price_per_unit
Count
purchased_price_per_unit

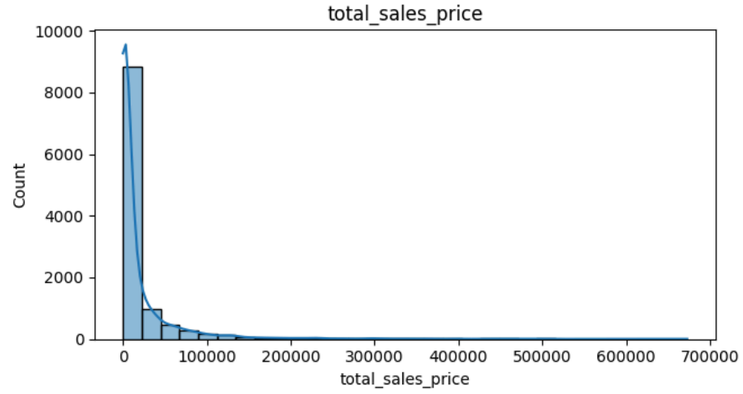

total_qty_purchased
Count
total_qty_purchased

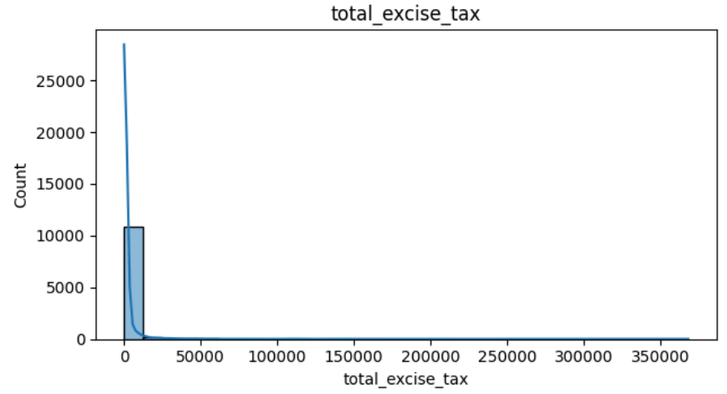

total_purchases_amount
Count
total_purchases_amount
1e6


selling_price_per_unit
Count
selling_price_per_unit


total_qty_sold
Count
total_qty_sold


total_sales_amount
Count
total_sales_amount
1e6

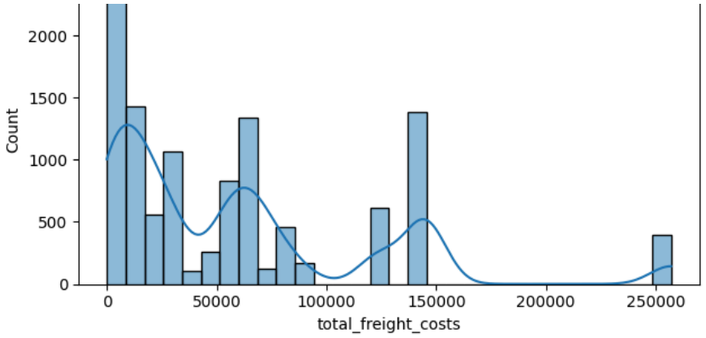

total_sales_price
Count
total_sales_price

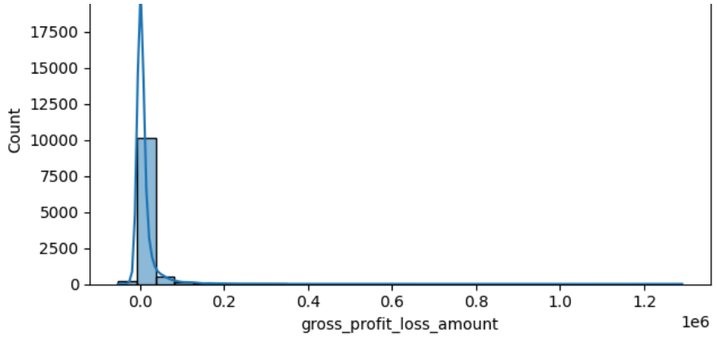

total_excise_tax
Count
total_excise_tax

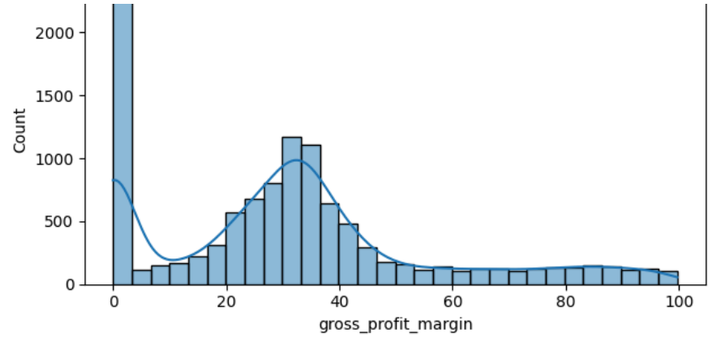
total_freight_costs

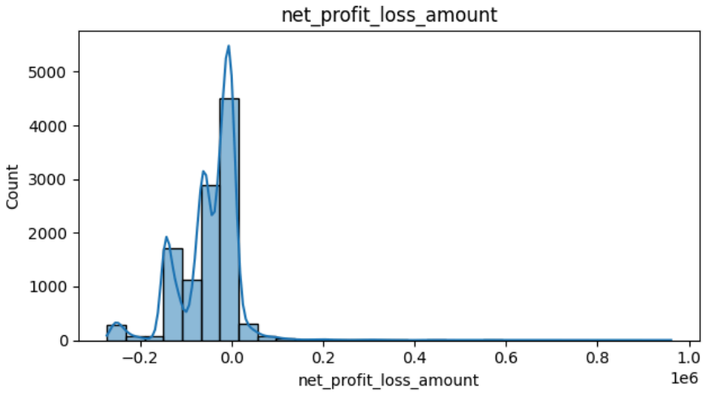
gross_profit_loss_amount

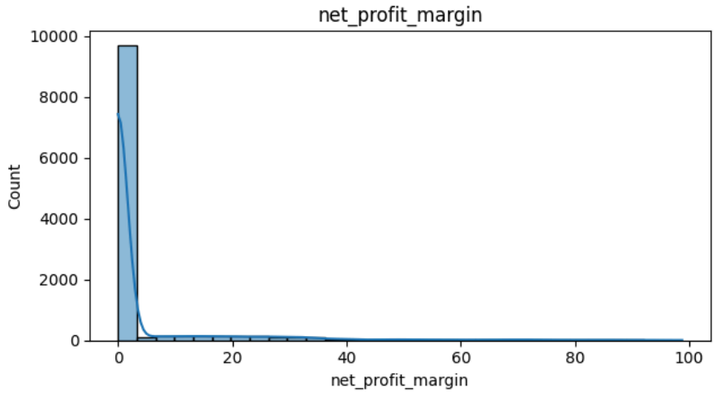
gross_profit_margin

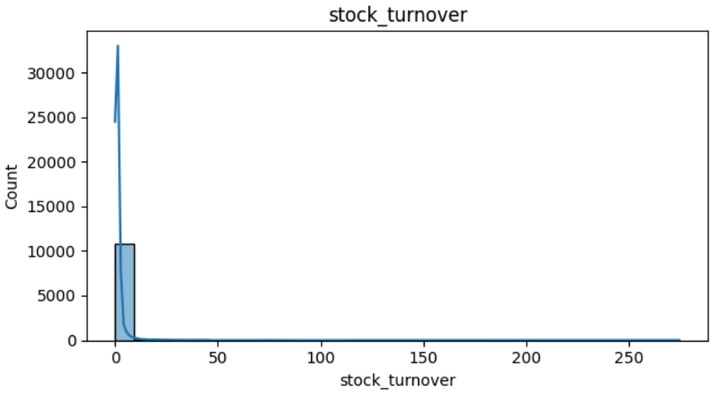


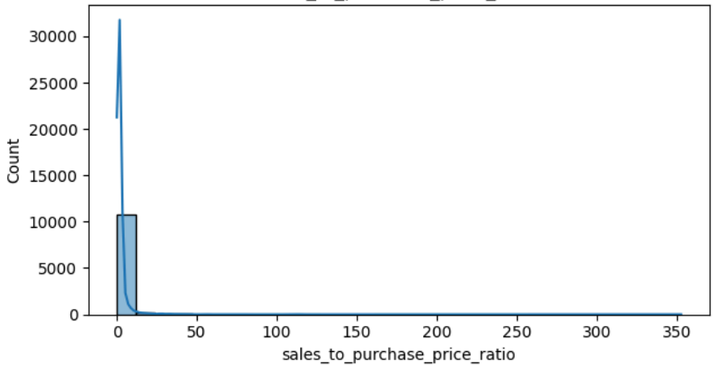




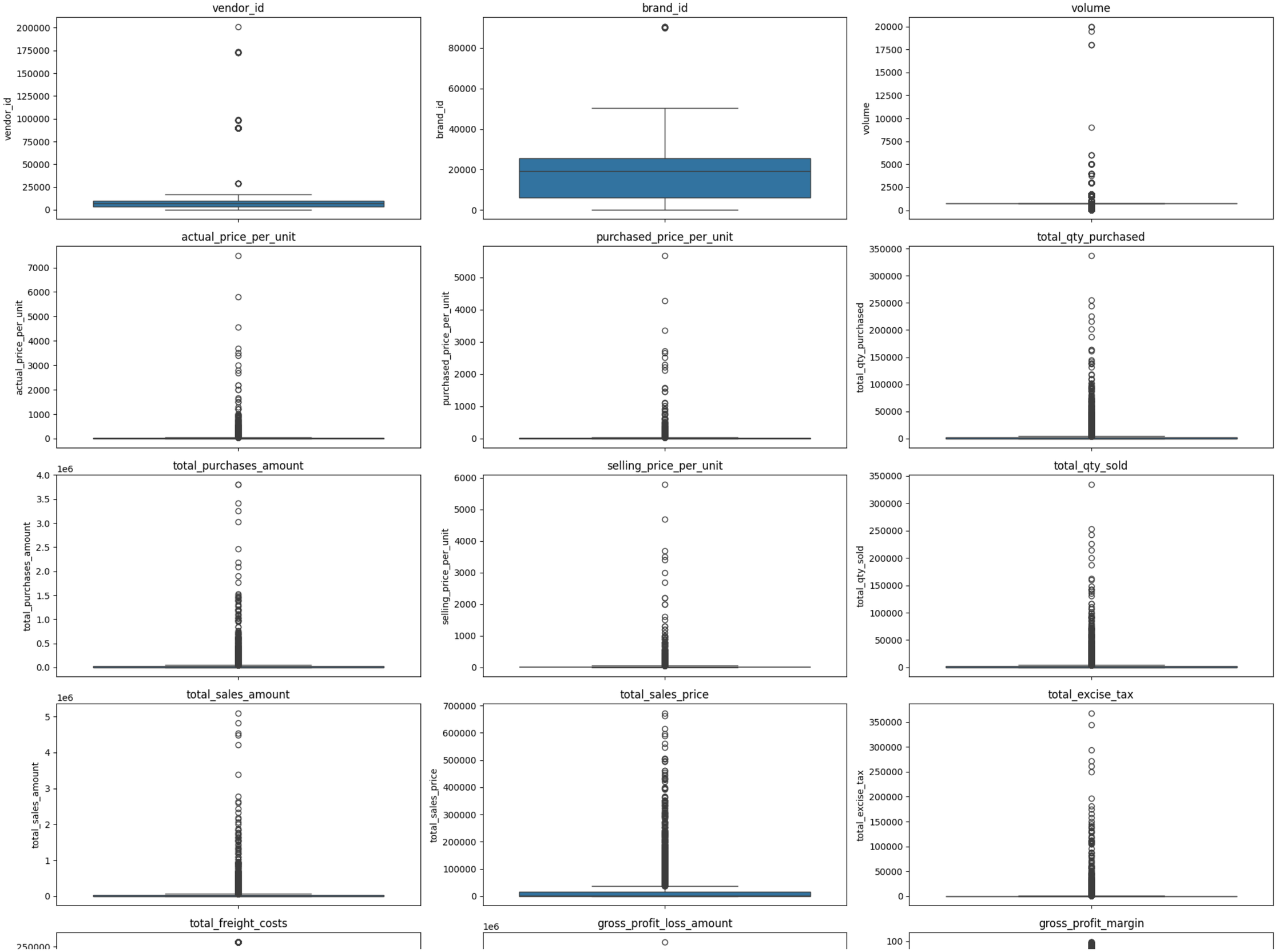


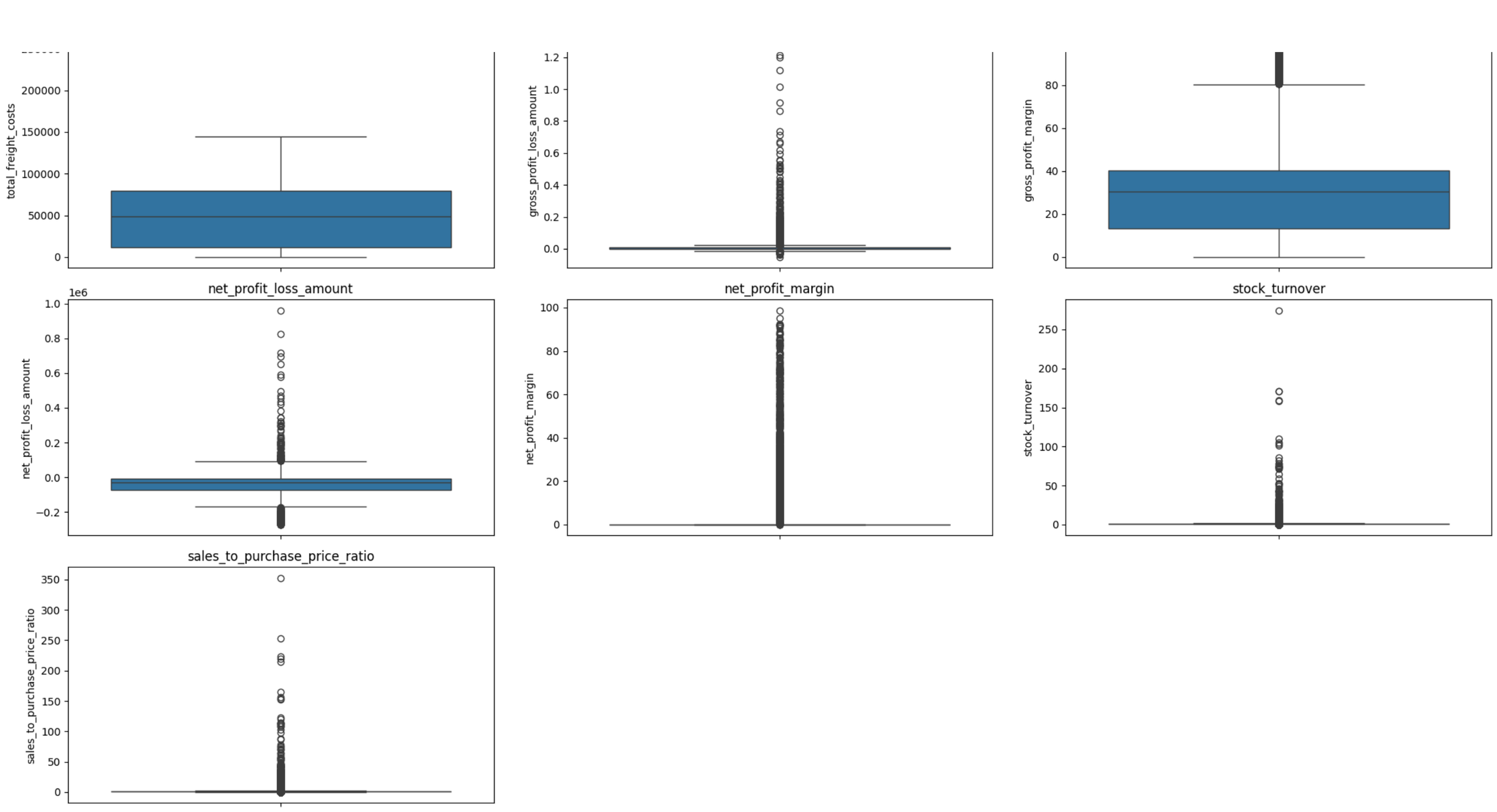






In [59]:
```
plt.figure(figsize=(20,25))

for i, col in enumerate(numerical_cols):
    plt.subplot(7, 3, i+1)
    sns.boxplot(y = supplier_performance[col])
    plt.title(col)

plt.tight_layout()
plt.show()
```


vendor_id
brand_id
volume
actual_price_per_unit
purchased_price_per_unit
total_qty_purchased
total_purchases_amount
selling_price_per_unit
total_qty_sold
total_sales_amount
total_sales_price
total_excise_tax
total_freight_costs
gross_profit_loss_amount
gross_profit_margin



SUMMARY STATISTICS INSIGHTS

- **Purchase & Actual Prices:** Maximum purchase (5,681.81) and sales prices (7,499.99) are far above the mean (24.39 & 35.64), suggesting the presence of premium or high-value products.
- **Total Quantity Purchased & Sold:** Extremely wide range (1 to 337,660 purchased; 0 to 334,939 sold) indicates both bulk transactions and very low-volume items.
- **Total Sales Quantity & Price:** Minimum values are 0, meaning some products were purchased but never sold. These could be slow-moving or obsolete stock.
- **Excise Tax:** Ranges from 0 to 368,242.80 — implying that tax is heavily volume-driven or varies by product category.
- **Freight Cost:** Highly variable (0.09 to 257,032.07), which may reflect logistics inefficiencies, inconsistent shipping methods, or batch-level freight entries.
- **Gross Profit:** Minimum value is -52,002.78, indicating losses. Some products or transactions may be selling at a loss due to high costs or selling at discounts lower than the purchase price.
- **Gross Profit Margin:** A minimum of 0% suggests some products generate no revenue, while 75% of vendors have GPM ≤ 40.20%, showing moderate profitability across most suppliers.

- **Net Profit Margin:** Much lower than GPM — with 75% of vendors having NPM = 0% — showing that expenses like freight and excise tax significantly reduce net profits.
- **Stock Turnover:** Ranges from 0 to 274.5, implying some products sell extremely fast while others remain in stock indefinitely. Value more than 1 indicates that quantity sold for that product is higher than purchased quantity due to either sales are being fulfilled from older stock.

Therefore, to ensure consistency and meaningful analysis, we will filter the dataset to include only those suppliers with both sales and gross profit margin greater than zero.

In [75]:
```
supplier_performance.shape
```

Out[75]:
```
(11101, 23)
```

In [118…
```
supplier_performance = supplier_performance[(supplier_performance['total_qty_sold'] > 0) & (supplier_performance['gross_profit_loss_amount'] > 0) & (supplier_performance['gross_profit_margin'
```

In [119…
```
supplier_performance.shape
```

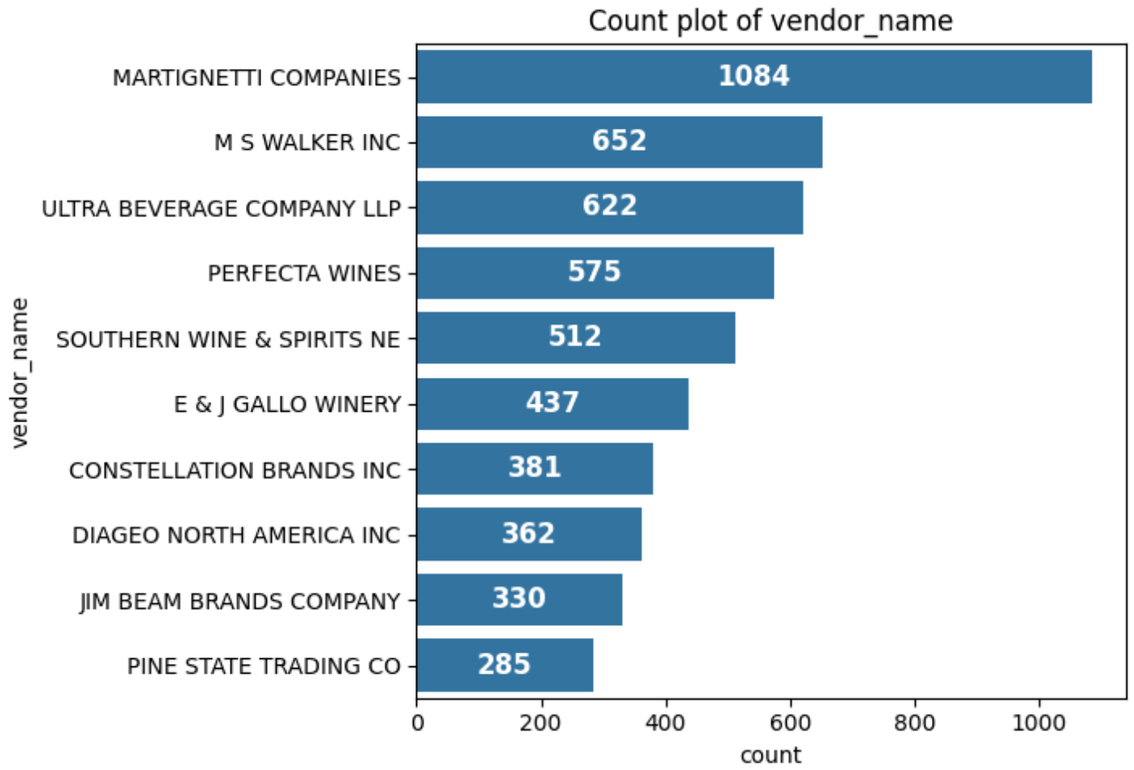
Out[119…
```
(8890, 23)
```

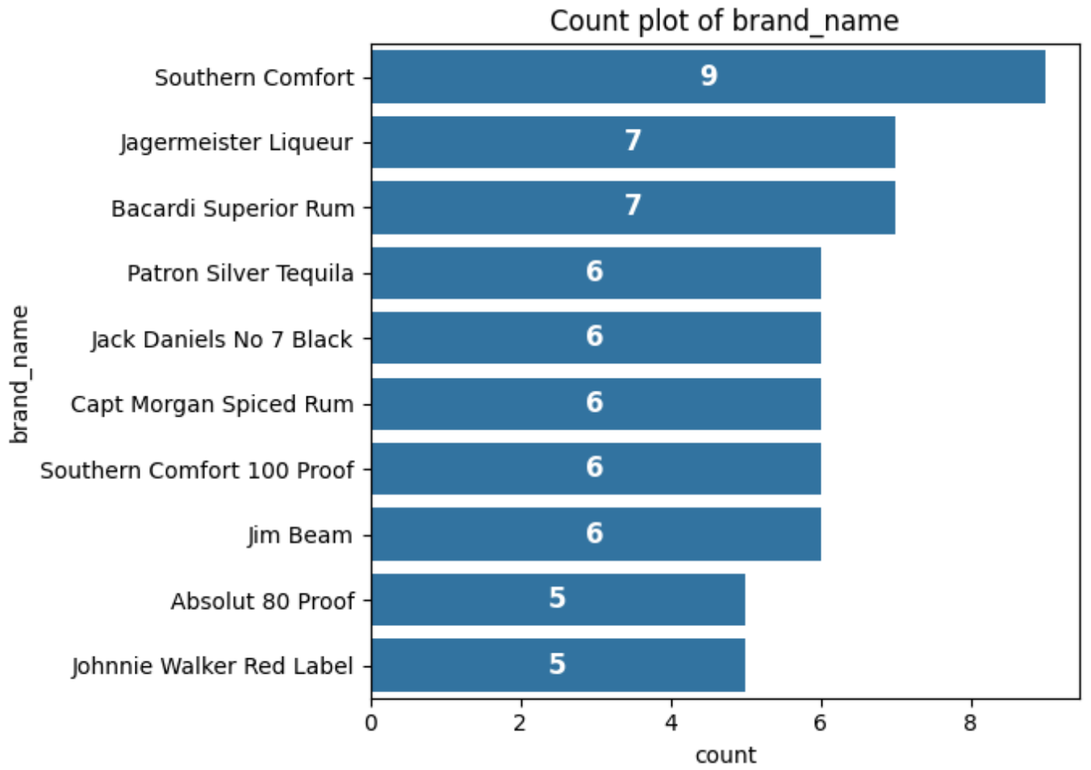
Our Dataset is Filtered.

In [120…
```
# To get only categorical columns
categorical_col = supplier_performance.select_dtypes(include='object').columns
categorical_col
```

Out[120…
```
Index(['vendor_name', 'brand_name', 'gross_profit_loss_status',
       'net_profit_loss_status'],
      dtype='object')
```

In [163…
```
cat = ["vendor_name", "brand_name"]

plt.figure(figsize=(14,5))

for i, col in enumerate(cat):
    plt.subplot(1,2, i+1)
    ax = sns.countplot(y=supplier_performance[col], order=supplier_performance[col].value_counts().index[:10])
    plt.title(f"Count plot of {col}")

    for bars in ax.containers:
        ax.bar_label(bars, label_type='center', color = 'white', fontsize = 12, fontweight = 'bold')

plt.tight_layout()
plt.show()
```



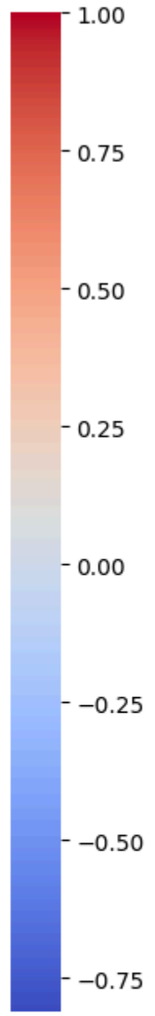


In [178…

```
plt.figure(figsize=(15,8))
corr_matrix = supplier_performance[numerical_cols].corr()
sns.heatmap(corr_matrix, annot=True, fmt = ".2f", cmap="coolwarm", linewidths=0.5, linecolor='black')

plt.title("Correlation Heatmap")
plt.show()
```

Correlation Heatmap

	vendor_id	brand_id	volume	actual_price_per_unit	purchased_price_per_unit	total_qty_purchased	total_purchases_amount	selling_price_per_unit	total_qty_sold	total_sales_amount	total_sales_price	total_excise_tax	total_freight_costs	gross_profit_loss_amount	gross_profit_margin	net_profit_loss_amount	net_profit_margin	stock_turnover	sales_to_purchase_price_ratio
vendor_id	1.00	0.07	0.00	-0.02	-0.02	-0.06	-0.04	-0.02	-0.06	-0.05	-0.07	-0.03	-0.19	-0.06	-0.02	0.14	0.09	-0.01	-0.00
brand_id	0.07	1.00	-0.00	-0.02	-0.03	-0.14	-0.15	-0.02	-0.14	-0.15	-0.25	-0.16	-0.18	-0.13	0.07	0.09	0.09	-0.02	-0.01
volume	0.00	-0.00	1.00	0.03	0.03	0.01	0.10	0.03	0.01	0.10	0.14	0.13	-0.01	0.10	-0.06	0.06	0.00	-0.04	-0.04
actual_price_per_unit	-0.02	-0.02	0.03	1.00	1.00	-0.05	-0.02	1.00	-0.05	-0.02	-0.02	-0.02	-0.02	-0.02	0.05	0.00	0.03	0.01	0.00
purchased_price_per_unit	-0.02	-0.03	0.03	1.00	1.00	-0.05	-0.01	1.00	-0.05	-0.01	-0.01	-0.02	-0.01	-0.02	0.04	-0.00	0.03	0.00	0.00
total_qty_purchased	-0.06	-0.14	0.01	-0.05	-0.05	1.00	0.67	-0.05	1.00	0.69	0.55	0.62	0.17	0.70	-0.12	0.24	0.08	-0.05	-0.05
total_purchases_amount	-0.04	-0.15	0.10	-0.02	-0.01	0.67	1.00	-0.02	0.67	1.00	0.82	0.88	0.16	0.96	-0.12	0.40	0.09	-0.04	-0.04
selling_price_per_unit	-0.02	-0.02	0.03	1.00	1.00	-0.05	-0.02	1.00	-0.05	-0.02	-0.02	-0.02	-0.01	-0.02	0.05	0.00	0.03	0.01	0.00
total_qty_sold	-0.06	-0.14	0.01	-0.05	-0.05	1.00	0.67	-0.05	1.00	0.69	0.55	0.62	0.17	0.70	-0.11	0.24	0.09	-0.04	-0.05
total_sales_amount	-0.05	-0.15	0.10	-0.02	-0.01	0.69	1.00	-0.02	0.69	1.00	0.83	0.87	0.15	0.98	-0.11	0.42	0.10	-0.04	-0.04
total_sales_price	-0.07	-0.25	0.14	-0.02	-0.01	0.55	0.82	-0.02	0.55	0.83	1.00	0.66	0.20	0.81	-0.18	0.29	0.10	-0.07	-0.07
total_excise_tax	-0.03	-0.16	0.13	-0.02	-0.02	0.62	0.88	-0.02	0.62	0.87	0.66	1.00	0.13	0.82	-0.09	0.30	0.04	-0.03	-0.03
total_freight_costs	-0.19	-0.18	-0.01	-0.02	-0.01	0.17	0.16	-0.01	0.17	0.15	0.20	0.13	1.00	0.15	-0.03	-0.81	-0.29	0.00	-0.00
gross_profit_loss_amount	-0.06	-0.13	0.10	-0.02	-0.02	0.70	0.96	-0.02	0.70	0.98	0.81	0.82	0.15	1.00	-0.07	0.45	0.14	-0.04	-0.04
gross_profit_margin	-0.02	0.07	-0.06	0.05	0.04	-0.12	-0.12	0.05	-0.11	-0.11	-0.18	-0.09	-0.03	-0.07	1.00	-0.01	0.19	0.41	0.42
net_profit_loss_amount	0.14	0.09	0.06	0.00	-0.00	0.24	0.40	0.00	0.24	0.42	0.29	0.30	-0.81	0.45	-0.01	1.00	0.36	-0.02	-0.02
net_profit_margin	0.09	0.09	0.00	0.03	0.03	0.08	0.09	0.03	0.09	0.10	0.10	0.04	-0.29	0.14	0.19	0.36	1.00	0.05	0.05
stock_turnover	-0.01	-0.02	-0.04	0.01	0.00	-0.05	-0.04	0.01	-0.04	-0.04	-0.07	-0.03	0.00	-0.04	0.41	-0.02	0.05	1.00	1.00
sales_to_purchase_price_ratio	-0.00	-0.01	-0.04	0.00	0.00	-0.05	-0.04	0.00	-0.05	-0.04	-0.07	-0.03	-0.00	-0.04	0.42	-0.02	0.05	1.00	1.00



CORRELATION INSIGHTS

- **Actual & Selling Price VS Total Sales:** The actual price and selling price have weak correlation with total sales (-0.02, -0.02). This shows price variations between actual price and selling price do not impact the total sales revenue.
- **Purchase VS Selling Prices:** Purchase Price has a strong correlation with Selling Price (+1) indicating that as the purchasing price increases, the selling price also increases.
- **Total Quantity Purchased VS Sold:** There is a very strong correlation between them (+1), confirming efficient inventory turnover.
- **Excise Tax:** Total excise tax has a strong correlation with total purchases (0.88), total sales (0.87) and gross profit / loss amount (0.82) indicating that the purchases, sales and gross profit increases, the overall excise tax to be paid also increases.
- **Freight Cost:** Total freight cost has a negative correlation with net profit / loss amount (-0.81), indicating that higher freight costs lead to decrease in net profit.
- **Gross Profit Margin:** GPM has a slight negative correlation with total sales price (-0.11) indicating that GPM decreases slightly with increase in total sales price. This could be due to competitive pricing in the market.
- **Stock Turnover:** Stock Turnover has a weak correlation with net profit margin (0.05) indicating that faster turnover doesnot necessarily result in higher profitability.
